UNITÀ 01

LEZIONE 03

Le funzioni in Python

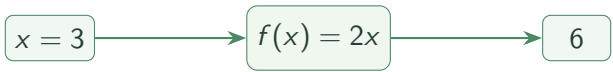
Parametri, chiamate e risultati

Prof. Francesco Gobbi
Anno scolastico 2026/2027
I.I.S.S. Galileo Galilei – Ostiglia (MN)

Il concetto di funzione

Una funzione raccoglie istruzioni per svolgere un compito. Può ricevere dati e restituire un risultato.

In matematica, la funzione $f(x) = 2x$ associa a ogni valore di ingresso il suo doppio.



In un programma, una funzione può anche produrre un effetto, come una stampa. Il valore restituito e ciò che appare sullo schermo sono due aspetti distinti.

Le chiamate già utilizzate

Chiamata	Comportamento
`print(12)`	Mostra 12 e restituisce `None`.
`input("Numero: ")`	Legge una risposta e restituisce una stringa.
`range(1, 4)`	Crea un oggetto che rappresenta 1, 2, 3.

Le parentesi indicano una **chiamata**; al loro interno vengono forniti gli argomenti.

`print` e `input` sono funzioni incorporate. Precisamente, `range` è un tipo incorporato richiamabile: la sua chiamata crea un oggetto `range`.

Definizione, parametro e argomento

Il codice

```
def mostra(numero):
    print(numero)

valore = 7
mostra(valore)
```

Output: 7

Le ultime due righe sono qui eseguite al livello del modulo. Nelle slide successive saranno raccolte in `main()`.

Definizione
`def mostra(numero):` e il corpo indentato definiscono la funzione; `mostra` ne è il **nome**.

Parametro: numero
È il nome locale che riceve il riferimento al dato della chiamata.

Corpo: print(numero)
È l'istruzione eseguita quando la funzione viene chiamata.

Chiamata: mostra(valore)
`valore` è l'**argomento**: la sua valutazione fornisce l'oggetto 7.

Il programma organizzato in main()

```
def main():
    # Corpo del programma principale
    print(7)

if __name__ == '__main__':
    main()  # Avvio del programma
```

def definisce una funzione. Il corpo indentato di `main()` contiene le istruzioni principali.

Quando il file viene eseguito direttamente, `__name__` vale `'__main__'`: il controllo finale chiama `main()`.

Output: 7.

Dal modulo all'esecuzione di main()

1. Il file viene eseguito

Il codice esterno alle funzioni è **codice del modulo**. Le istruzioni `def` creano le funzioni, senza eseguirne il corpo.

2. Il controllo finale permette l'avvio

```
if __name__ == '__main__':
    main()
```

Il controllo è vero quando il file viene eseguito direttamente.

3. La chiamata esegue il corpo di main()

Le istruzioni principali sono raccolte esplicitamente in `main()`, un nome scelto **per convenzione**.

Durante un'importazione, il controllo finale è falso: la chiamata a `main()` non viene avviata da quel blocco.

La prima funzione: stampare un numero

```
def mostra(numero):
    print(numero)  # Dato ricevuto

def main():
    valore = 7  # Nome locale di main
    mostra(valore)  # Chiamata

if __name__ == '__main__':
    main()  # Avvio
```

Definizione
`def mostra(numero):` introduce la funzione e il suo corpo.

Parametro: numero
È il nome locale usato nel corpo per riferirsi al dato ricevuto.

Argomento: valore
Nella chiamata, il nome `valore` viene valutato: il suo oggetto viene associato a `numero`.

Output: 7.

Lo schema di definizione e chiamata

Definizione

```
def nome_funzione(parametri):
    istruzioni del corpo
```

Chiamata

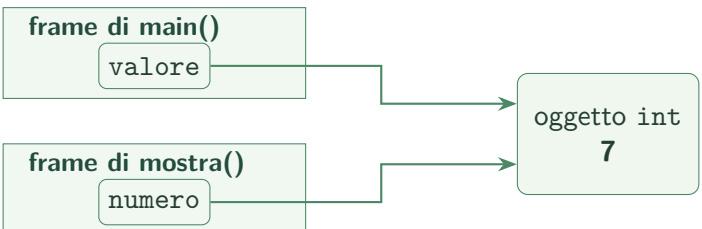
```
nome_funzione(argomenti)
```

Nel caso precedente: nome `mostra`, parametro `numero`, argomento `valore`. La funzione deve essere definita prima che la chiamata venga eseguita.

Nota bene – indentazione. Il rientro definisce il corpo della funzione. Negli esempi vengono usati quattro spazi. Il ritorno al margine conclude il blocco della definizione.

Una chiamata, due riferimenti allo stesso oggetto

`mostra(valore)` associa il **parametro numero** allo stesso oggetto indicato dall'argomento `valore`.



Ogni freccia parte da un **nome** e arriva all'**oggetto**. Viene creato un nuovo nome locale, senza copiare automaticamente l'oggetto. In Python si parla di **riferimenti**.

Lo stack: chiamata e ritorno

Ogni chiamata attiva ha un **frame** con i propri nomi locali. Il frame della funzione chiamata viene aggiunto sopra quello del chiamante.



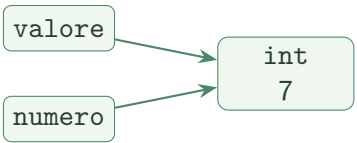
Durante: `main()` attende; `mostra()` esegue il proprio corpo.

Dopo: il frame di `mostra()` termina e `main()` riprende dall'istruzione successiva alla chiamata.

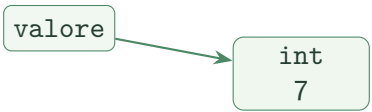
Le frecce tra le fasi indicano il flusso di esecuzione; quelle accanto ai nomi indicano riferimenti.

Al ritorno, che cosa accade alle frecce?

Durante la chiamata



Dopo il ritorno



il frame di `mostra()` è terminato

Nel semplice esempio, il riferimento locale `numero` scompare con la fine della chiamata. Il nome `valore`, in `main()`, continua a riferirsi all'oggetto.

Il ritorno da una funzione non elimina automaticamente gli oggetti usati: altri riferimenti possono mantenerli raggiungibili.

Riassegnare un parametro

```
def aumenta(numero):
    numero = numero + 1  # Nuovo legame
    print(numero)  # Mostra 8

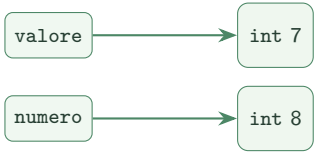
def main():
    valore = 7
    aumenta(valore)
    print(valore)  # Mostra ancora 7

if __name__ == '__main__':
    main()
```

Prima dell'assegnazione



Dopo numero = numero + 1



Cambia solo il riferimento del **parametro locale**. L'intero 7 resta invariato.

Output: 8, poi 7.

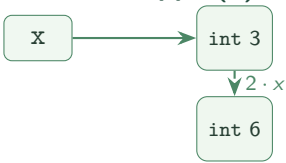
Dal modello matematico a return

```
def doppio(x):
    return 2 * x  # Risultato restituito

def main():
    risultato = doppio(3)
    print(risultato)  # Mostra 6

if __name__ == '__main__':
    main()
```

Durante doppio(3)



Dopo il ritorno in main()



`return` restituisce l'oggetto risultato; l'assegnazione lo associa a **risultato**. Il frame di `doppio()` termina.

Freccia continua: **riferimento**. Tratteggiata: **calcolo**.

Stampare e restituire un valore

```
def mostra(numero):
    print(numero)  # Effetto sullo schermo

def main():
    esito = mostra(7)
    print(esito)  # Mostra None

if __name__ == '__main__':
    main()
```

La prima stampa mostra 7.

In assenza di un `return` esplicito, una funzione termina restituendo `None`.

Quindi `esito` fa riferimento a `None`, non a 7.

Un valore da usare nei calcoli deve essere restituito con `return`.

Funzione 1: la sommatoria

```
def sommatoria(n):  # n intero >= 0
    totale = 0  # Accumulatore
    for numero in range(1, n + 1):
        totale = totale + numero
    return totale  # Dopo il ciclo

def main():
    somma = sommatoria(4)
    print(somma)  # Mostra 10

if __name__ == '__main__':
    main()
```

Compito: calcolare $1 + 2 + \cdots + n$.

`n` è il parametro; 4 è l'argomento. `totale` e `numero` sono nomi locali della funzione.

Il totale passa da 0 a 1, 3, 6, 10.

`return` è fuori dal ciclo: restituisce la somma completa.

Sommatoria: ingresso, lavoro e risultato

Chiamata	Calcolo	Risultato
`sommatoria(4)`	$1 + 2 + 3 + 4$	10
`sommatoria(1)`	1	1
`sommatoria(0)`	Nessuna iterazione	0

Con `n = 0`, `range(1, 1)` è vuoto: il totale iniziale viene restituito senza modifiche.

Precondizione: l'argomento è un intero non negativo. Gli esempi assumono dati validi; non eseguono ancora la validazione dell'ingresso.

Funzione 2: il fattoriale

```
def fattoriale(n):  # n intero >= 0
    prodotto = 1  # Elemento neutro
    fattore = 1  # Primo fattore
    while fattore <= n:
        prodotto = prodotto * fattore
        fattore = fattore + 1
    return prodotto  # Prodotto finale

def main():
    print(fattoriale(4))  # Mostra 24

if __name__ == '__main__':
    main()
```

Compito: calcolare $n! = 1 \cdot 2 \cdots n$.

Il prodotto parte da 1; con partenza da zero resterebbe zero.

Con 4, i prodotti parziali sono 1, 2, 6, 24.

Con 0, il corpo viene saltato: viene restituito 1, coerentemente con $0! = 1$.

Funzione 3: il maggiore di due numeri

```
def maggiore(a, b):
    if a >= b:  # Primo valore maggiore o uguale
        return a  # La chiamata termina qui
    return b  # Caso a < b

def main():
    massimo = maggiore(8, 5)
    print(massimo)  # Mostra 8

if __name__ == '__main__':
    main()
```

Due parametri: a e b.

Gli argomenti posizionali vengono associati in ordine: 8 ad a, 5 a b.

Il primo `return` eseguito termina la funzione. Se a < b, viene raggiunto il secondo.

A parità di valore viene restituito a.

Tre funzioni, tre risultati

Funzione	Chiamate di prova	Risultati
Somma da 1 a n	`sommatoria(5)`, `sommatoria(0)`	15; 0
Prodotto da 1 a n	`fattoriale(5)`, `fattoriale(0)`	120; 1
Maggiore di due valori	`maggiore(-2, -7)`, `maggiore(3, 3)`	-2; 3

Ogni chiamata crea un nuovo frame con i propri nomi locali. L'accumulatore viene inizializzato di nuovo a ogni chiamata.

Il chiamante fornisce gli argomenti e riceve il risultato; i nomi temporanei usati per il calcolo appartengono alla funzione chiamata.

Lettura dei dati nel programma principale

```
def doppio(x):
    return 2 * x  # Calcolo riutilizzabile

def main():
    numero = int(input("Numero: "))
    risultato = doppio(numero)
    print("Doppio:", risultato)

if __name__ == '__main__':
    main()  # Avvio del programma
```

`main()` gestisce lettura e stampa. La funzione riceve un numero già acquisito.

Con ingresso 5, il parametro `x` e il nome `numero` indicano inizialmente lo stesso oggetto.

Output: `Doppio: 10`.

La stessa funzione può essere richiamata anche con un valore scritto direttamente nel codice.